Executable Requirement Memory for LLM Coding Agents Under Changing Requirements: An Open-Data Study with a Local Open-Weight Model

Pankaj Kumar*

October 2026

Abstract

An LLM coding agent that works across many sessions must carry earlier requirements forward, and those requirements change. One proposal is an *executable* memory: compile each requirement into tests and replay an append-only ledger of them on every change. The risk is that the ledger calcifies, so that stale tests block correct new work. We run LedgerBench, a benchmark with a frozen, pre-registered 24-task sequence containing two requirement regime changes, five trap tasks and a planted defective test, against five memory strategies with 12 independent runs each (60 runs), using a locally served open-weight model (Qwen3-Coder-30B-A3B). Every run log is public, and every number in this paper is regenerated from those logs by one command. Three results are robust under exploratory nonparametric tests with multiplicity correction. (1) Every ledgered arm makes the hidden-oracle-correct change more often than a summary-only baseline (per-task pass rate 0.698–0.750 versus 0.597). (2) Every ledgered arm commits fewer violations of earlier requirements than the baseline, by roughly half to two-thirds per run, and the ledgered arms do not differ from one another once violations committed under stale-test pressure are counted. (3) The plain append-only ledger costs 2.44× the baseline's tokens, while evidence-governed variants cost 61–65% of the plain ledger. The pre-registered primary endpoint, cumulative success at the final task, shows no detectable difference between arms (Kruskal–Wallis $p = 0.554$): seed variance dominates. Of the pre-registered confirmatory predictions, H1-trap holds for two of four ledgered arms and H1-rewrite for all four, but the plain ledger's clean H1 record depends entirely on the pre-registered exclusion of coerced violations. We report one protocol deviation, made before the main campaign, which tells the agent the project's file names; without it the agent could not locate the code.

1 Introduction

Real software changes its mind. A requirement settled one week is overturned the next: "store tasks in a JSON file" becomes "store them in SQLite." An LLM coding agent working across sessions has no durable memory of its own, so some mechanism must carry the accumulated requirements forward. A natural mechanism is an *executable* memory: compile each requirement into automated tests, keep an append-only ledger of them, and replay the ledger on every proposed change, so that a requirement once satisfied is never silently dropped.

*Independent researcher. Correspondence: `contact@nordneuron.com`. Code, frozen task sequence, pre-registration, every run log, and the script that regenerates every number in this paper: `https://github.com/NordNeuron/ledgerbench`; archived at doi:10.5281/zenodo.23450108.

The risk is the mirror image of the benefit. When a requirement is overturned, the tests that encode it do not become irrelevant; they are about exactly the thing that changed, and they fail against any implementation of the new requirement. An agent gated on the whole ledger may then be unable to make correct progress, so that the memory meant to keep it honest instead *calcifies*. Governance can help: an isolated judge could retire tests that documented evidence shows to be stale. But every retirement also removes a guard.

LedgerBench isolates this tension with a frozen 24-task sequence on a small command-line application, five memory strategies, and hidden oracles that score correctness independently of whatever tests the agent is held to. An earlier version of this study was run against a hosted model; its run logs were lost when the machine that held them failed, so none of its numbers can be verified and we do not report them. This paper is a complete re-execution of the frozen protocol on a local open-weight model, with every log committed.

Contributions:

1. An **open-data** execution of a pre-registered benchmark for agent memory under changing requirements. All 60 run logs (every prompt, response, verification and oracle result), the workspaces' full git histories, and a script that regenerates each of the numbers in this paper are public (Section 8).

2. Evidence, with exploratory inferential statistics, that executable requirement ledgers improve per-task correctness and roughly halve requirement violations relative to a summary-only memory, at a large token cost that evidence-governed retirement reduces by about a third (Section 5).

3. A candid pre-registered scorecard, including refuted predictions and a sensitivity analysis showing that one pre-registered scoring rule, coercion exclusion, determines the plain ledger's apparent perfection (Section 6).

4. Practical lessons for running long-horizon agent benchmarks on local models: silent context truncation and an unstated file layout each corrupted early attempts, and we document how we detected and handled both (Section 4).

2 Related work

Benchmarks for coding agents. SWE-bench [1] evaluates whether models resolve real GitHub issues, one issue at a time, against a fixed repository state. LedgerBench differs in two ways: tasks arrive as a *sequence* on one evolving codebase, and requirements stated early must survive later work, including deliberate changes of regime.

Memory for LLM agents. Generative agents [2] maintain a natural-language memory stream with retrieval and reflection; MemGPT [3] pages information between a bounded context and external storage; Reflexion [4] carries verbal self-feedback across attempts. Long-term conversational memory benchmarks such as LoCoMo [5] test recall over long dialogues, and long-context studies show that models use information unevenly across a long prompt [6]. Learned retrieval over memory graphs, such as HAGE [7], optimizes which memories to surface. These lines share an assumption that the stored facts stay true. LedgerBench's question is what happens when they *expire*: our Arm A is a summary memory in the spirit of reflection-based approaches, and Arms B–D are executable memories with different retirement rules.

Regression testing and test evolution. Replaying accumulated tests on every change is regression testing [8]; requirement-to-test links are a form of requirements traceability [9]; and keeping tests in step with intended behaviour is the test-repair problem [10]. Test-driven practice [11] makes tests the specification. Our ledger is an automated, agent-facing version of this practice, and our stale-test failure mode is the familiar obsolete-test problem seen from the agent's side.

Specification gaming. When an optimized system can influence the measure it is judged by, it may improve the measure without improving the target [12]. Our Arm D lets the agent ask to retire its own tests, a discrete and auditable version of that channel.

Pre-registration. Freezing hypotheses and analysis rules before seeing data separates confirmatory from exploratory claims [13]. We re-use the original frozen rules unchanged and label every non-registered analysis as exploratory.

3 Benchmark design

3.1 Task sequence

The seed project, `taskcli`, is a plain standard-library Python command-line task manager (`add`, `list`, `done`, `delete`). A fixed sequence of 24 tasks evolves it in sessions of three tasks; context is cleared at each session boundary. The sequence contains:

- **Regime changes.** A storage migration (JSON file to SQLite) at T6, a contradictory schema change (table `tasks` to `items`) at T14, and a merely-obsolete change (`config.toml` replacing environment variables) at T20. Each ships an *evidence artifact* naming the assertion assumptions it invalidates.
- **Trap tasks.** 5 non-adjacent tasks (T15, T17, T19, T21, T23) whose natural implementation tempts the agent to break an earlier, still-binding requirement. Five *primary* requirements, fixed by a calibration criterion before registration, carry the confirmatory tests: silent success (R9), insertion-order default (R2-order), the exit-code contract (R11), atomic writes for file outputs (R10) and pipe safety (R12). Two secondary requirements (R13, R2-ids) are exploratory.
- **A planted defective assertion** (A11f) that encodes an error-message format no prompt requires, and a **final conflict** (T24) that is jointly unsatisfiable with an earlier requirement and has no evidence on either side.

Each task has 2–5 reference assertion tests (what the agent may be held to) and 2–6 hidden oracle tests (53 in total) that probe the same requirements end-to-end and remain valid across migrations. The oracles are never shown to the agent.

3.2 The five arms

All arms run the same model on the same tasks; they differ in what memory the agent carries and how a proposed change is verified.

A — Summary baseline. A rolling model-written summary across sessions; each task is verified only against its own reference assertions.

B — Append-only ledger. Every admitted assertion is replayed on every change and never removed; the assertion texts are the carried context.

C — Evidence-governed ledger. B plus an isolated judge (a separate conversation) that reviews assertions flagged by an evidence artifact, and escalates admission-time conflicts, and may retire an assertion to a log-only audit pool. Generator prompts never contain governance vocabulary or retired assertion content.

C+ — Governed + telemetry docket. C plus automatic judge review of any assertion that fails in the repair loops of at least two distinct tasks.

D — Governed + generator voice. C plus a channel through which the agent may request a test's retirement, with the judge shown the agent's justification; a retirement granted on that basis is scored as *capture*.

3.3 Metrics and frozen scoring rules

The pre-registered primary endpoint is **M1@T24**: the fraction of tasks whose hidden oracles pass on the final workspace. We also report the per-task oracle pass rate (each task's own oracle on its own final state), regressions (M2), blocked valid solutions (M4: the oracle passes but ledger replay fails; structurally zero in Arm A), governance activity (M6) and tokens (M7). Requirement violations are detected by standing oracle detectors and split into two pre-registered channels: **H1-trap**, a violation at the requirement's trap task, and **H1-rewrite**, a violation anywhere else. Two mechanical exclusions were frozen with the rules. A violation during a *broken window* (the workspace fails a scripted add/list smoke probe) is skipped. A violation is *coerced*, and excluded from H1, when the repair feedback at that task contained a failing assertion invalidated by evidence. Violations at the T24 conflict are reported separately.

The confirmatory hypotheses are that each ledgered arm commits at most one third of Arm A's H1-trap and H1-rewrite violations on the primary requirements. The sequence, oracles and these rules were frozen at commit `257dc61`.

4 Experimental setup

Model and serving. All generator, summary and judge calls use Qwen3-Coder-30B-A3B-Instruct [14], a mixture-of-experts coding model with about 3B active parameters, in the Ollama `qwen3-coder:30b` build (Q4_K_M quantization), served by Ollama 0.34.4 through its OpenAI-compatible endpoint. The served variant (`qwen3-coder-128k`) sets a 131,072-token context window; the key–value cache was quantized to 8 bits with flash attention enabled so that the model and full context fit in GPU memory. Temperature was 0.2 and the per-call output limit was 8000 tokens; the other sampling parameters were the build's defaults. Hardware was a single Apple M5 Max Mac Studio with 36 GB of unified memory.

Runs. Five arms $\times$ 12 seeds = 60 runs, run sequentially. The harness does not pass the seed to the model server, so seeds are independent repeats rather than reproducible random streams; the committed logs, not a rerun, are the evidence. Repair budget $k = 4$.

Frozen inputs. The task definitions, reference assertions and oracles used are byte-identical to the freeze commit. The scoring code's H1 rules and metric definitions are unchanged since the

freeze; later edits only corrected the two isolation scans (logged in July 2026, before this campaign) and made output paths repo-relative.

Protocol deviation: project layout. The generator must return complete files but never sees the workspace, and the seed README never names the source files. In a two-task smoke test under the published protocol the model wrote a new top-level `taskcli.py`, which `python -m taskcli` never runs because the seeded `taskcli/` package shadows it, and both tasks failed every repair iteration for that reason alone. We therefore enabled an opt-in flag that appends the list of source file *names* (not contents) to the README shown identically to every arm. The same smoke test then passed both tasks on the first iteration. The flag was set before the main campaign and the logs confirm that 3887 of 3887 generator calls carried it. This is a change to the registered protocol; it removes a path-guessing confound common to all arms and does not expose prior code, so the memory comparison is unchanged, but absolute success rates are not comparable to a run without it.

Excluded attempts and incidents. Before the final configuration, three partial attempts were discarded and are committed separately: one run under the published protocol (the layout problem above), one under a 32k context and five under a 64k context. The latter two were stopped because the server's logs showed prompts being truncated once repair loops grew; truncated prompts silently corrupt results. The main campaign used one configuration throughout; its largest single call (prompt plus output) was 86719 tokens, well inside the 131,072-token window, so no prompt could have been truncated. One run was interrupted by an unplanned machine restart and was rerun from scratch with the same seed (1 infrastructure rerun, as the registered rerun rule prescribes), and the campaign was once suspended for two hours with no loss of state. The registered no-peeking rule was not fully observed: during the campaign, progress checks displayed per-run pass counts and event counts. No scoring rule or configuration was changed after any such check; the analysis ran once, automatically, after the last run.

Isolation. An automated leak check over every logged prompt of all 60 runs found 0 occurrences of hidden-oracle content, and a separation-of-powers scan of all 36 governed runs found 0 violations.

Statistics. Inference was out of scope in the registration, so all tests here are exploratory. For each per-run endpoint we report bootstrap 95% confidence intervals (10,000 resamples, fixed seed), a Kruskal–Wallis test across arms, and two-sided Mann–Whitney tests in two families: each ledgered arm against A, and all pairs of ledgered arms. Each family is Holm-corrected, and we report Cliff's δ as the effect size.

5 Results

The primary endpoint does not separate the arms. Cumulative success at T24 ranges from 0.351 (D) to 0.569 (B) with standard deviations of 0.206–0.386 across runs; no comparison is significant (smallest Holm-corrected p against Arm A: 0.387; Table 2). The end state of a 24-task run is dominated by whether, and when, a run's workspace broke and recovered, and that varies more between seeds than between arms. With 12 seeds this endpoint is underpowered for effects of the size we observe.

Arm	M1@T24	per-task M1	own verdict	H1-trap	H1-rewrite	coerced	ktok/run
A (summary)	0.368	0.597	0.667	12	44	0	940
B (append-only)	0.569	0.719	0.198	0	0	27	2291
C (governed)	0.465	0.750	0.500	4	12	0	1414
C+ (gov. + docket)	0.420	0.743	0.531	11	13	0	1405
D (gov. + voice)	0.351	0.698	0.514	5	12	0	1479

Table 1: Main results, 12 runs per arm. M1@T24 and per-task M1 are hidden-oracle measures (means); "own verdict" is the pass rate under each arm's own verification and is *not* comparable across arms, because each arm verifies against different tests. H1 columns are totals over all runs on primary requirements under the frozen rules; coerced violations are excluded from H1-rewrite and listed separately. ktok/run is generator, summary and judge tokens (input + output) per run.

	vs. Arm A (Holm p, Cliff's δ)				KW p
Endpoint (per run)	B	C	C+	D	
M1@T24	0.387 (0.40)	1.000 (0.21)	1.000 (0.12)	1.000 (0.00)	0.554
Per-task M1	0.024 (0.60)	0.010 (0.73)	0.010 (0.71)	0.035 (0.51)	0.006
Regressions (M2)	0.086 (-0.49)	0.037 (-0.63)	0.078 (-0.54)	0.203 (-0.31)	0.062
H1 (frozen rules)	<0.001 (-1.00)	0.002 (-0.83)	0.003 (-0.71)	0.002 (-0.82)	<0.001
H1 incl. coerced	0.011 (-0.61)	0.002 (-0.83)	0.006 (-0.71)	0.002 (-0.82)	0.001
Tokens	<0.001 (1.00)	0.001 (0.79)	<0.001 (0.85)	<0.001 (0.90)	<0.001

Table 2: Exploratory tests, $n = 12$ per arm. Cliff's $\delta > 0$ means the ledgered arm's values tend to be larger than Arm A's. H1 counts are per-run trap plus rewrite violations on primary requirements.

Ledgers make the correct change more often. Judged by each task's own hidden oracle, every ledgered arm beats the baseline (0.698–0.750 versus 0.597; all four Holm-corrected comparisons significant, δ between 0.51 and 0.73). No pair of ledgered arms differs. Replaying earlier requirements' tests gives the agent concrete feedback that a summary does not.

Ledgers prevent requirement violations. Arm A commits 12 trap-channel and 44 rewrite-channel violations of primary requirements across its 12 runs, concentrated on insertion order (19), pipe safety (17) and exit codes (14). Every ledgered arm commits significantly fewer per run, whether or not coerced violations are counted (Table 2). Counting all violations (the "incl. coerced" row), the four ledgered arms are statistically indistinguishable from one another (all pairwise Holm $p = 1$), at roughly a third to a half of the baseline's per-run rate. Under the frozen rules, Arm B records zero violations and significantly fewer than every governed arm, but Section 5.1 shows that this difference is produced by the coercion rule rather than by B's behaviour.

The plain ledger is expensive; governance recovers a third. Arm B uses 2291k tokens per run, 2.44× Arm A's 940k, and takes 118 minutes per run against 43. Its ledger also rejects correct work: 369 repair iterations passed the hidden oracles (on the current task, with no newly broken earlier task) but failed ledger replay, 174 of them at T14 or later, and B's own verdict passes only 0.198 of tasks although its per-task oracle pass rate is 0.719. The governed arms cost 62%, 61% and 65% of B's tokens (C, C+, D) with indistinguishable correctness and violation rates, and block fewer correct iterations (222, 167, 154). Retiring what evidence shows to be stale is what buys this back.

5.1 Coercion decides the plain ledger's H1 record

The frozen coercion rule excludes a violation when the repair feedback at that task contained a failing evidence-invalidated assertion. In Arm B such assertions are never removed, so after the first regime change nearly every failing repair loop contains one. Apart from the T24 conflict, which is scored separately, every primary violation B committed (27) was excluded on this ground: 9 at trap tasks and 18 elsewhere. In the governed arms evidence-invalidated assertions are retired, and the rule never fired. The rule's rationale is sound: an agent forced to satisfy a contradictory stale test is not *forgetting* a requirement. But it means B's zeros say "B's violations happened under stale-test pressure," not "B never violated." Re-scoring with coerced violations restored to their channel, B's counts (9 trap, 18 rewrite) exceed both pre-registered thresholds (4 and 14.67), and B no longer differs from the governed arms.

5.2 Governance activity

Evidence-triggered retirement worked as designed: at T6 every governed run retired the JSON-storage assertions (C, C+, D: 12, 12 and 12 of 12 runs), at T14 every governed run retired the schema assertion A6 (12, 12 and 12 of 12 runs), and the other T14-stale assertion A10 was gone by T14 in every governed run, often already retired earlier through a conflict escalation. The admission-time conflict check was very active (234, 183 and 174 escalations in C, C+ and D) and the judge mostly retained the older assertion (203, 350 and 157 retain against 96, 88 and 85 supersede verdicts, all routes). The telemetry docket opened 191 reviews in C+, including the planted A11f in all 12 runs, but the judge retired A11f through the docket in only 2. The generator voice in Arm D was almost unused: 3 retirement requests in 12 runs, 1 of which named an active assertion, and the judge refused it, so capture was 0.

6 Pre-registered scorecard

Table 3 lists every frozen prediction. Three deserve comment.

The amendment-capable arms leak at the traps. C+ and D exceed the H1-trap threshold (11 and 5 against 4), while C meets it exactly. In C+ the excess is mostly silent success at its trap (6 of 11); in D it is mostly pipe safety (3 of 5). Away from the traps, pipe safety is also the requirement the governed arms violate most (9, 10 and 10 trap-plus-rewrite violations in C, C+ and D). These arms differ from C only in their amendment channels, so the observation is consistent with the mechanism registered for this study, that every retirement takes down a guard. But the differences between C, C+ and D are not significant (Table 2 reports no difference among ledgered arms), so we read this as a direction to test with more seeds, not as an established effect.

The plant was retired by the conflict path, not kept. We predicted that C would keep A11f forever, because no evidence artifact can flag an assertion with empty assumptions. Instead, the admission-time conflict check escalated A11f when a later task's assertions passed while it failed, and the judge retired it in 7 of 12 C runs. The prediction overlooked a second, evidence-free retirement route that the design itself contains.

Capture did not occur. The registered capture test depends on the agent using its voice; this model almost never did. The result is a null about this model's propensity to lobby, not evidence that the channel is safe.

Pre-registered prediction	Observed	Verdict
H1-trap: B, C, C+, D $\leq$ A/3 (A = 12, threshold 4)	B 0, C 4, C+ 11, D 5	B, C confirmed; C+, D refuted
H1-rewrite: B, C, C+, D $\leq$ A/3 (A = 44, threshold 14.67)	B 0, C 12, C+ 13, D 12	confirmed for all four
sensitivity, coerced restored (not registered)	B 9 / 18	B refuted on both
R10 violations are contract-grade	5 of 5 in A; none elsewhere	confirmed (yes)
Coercion fires only in B	B 27, others 0	confirmed
Plant A11f: C retains it in every run	retained in 5 of 12; 7 retired via conflict escalation	refuted
Plant A11f: C+ dockets it	docketed in 12 of 12, retired via docket in 2	confirmed (docketing)
D capture > 0; C capture = 0	D 0, C 0	D refuted; C confirmed
D late regressions (T19–T24) $\geq$ C	D 162, C 132	confirmed
T14: stale A6/A10 block valid work in B	174 blocked iterations at T14+	confirmed
T14: C-family retires A6 and A10	A6 retired at T14 in all governed runs; A10 gone by T14 in all	confirmed
T20: vestigial code B > C-family > A	95% / 93.3–100% / 93.3% of commits	undiscriminating (ceiling)
T24: B thrashes	12 of 12 B runs exhaust the budget	confirmed
T24: C-family escalates the conflict	C 8, C+ 5, D 5 of 12	partially

Table 3: Every pre-registered prediction against the observed data. The sensitivity row is not part of the registration.

7 Threats to validity and limitations

1. **One model, quantized, local.** All runs use one 4-bit quantized open-weight model with an 8-bit key–value cache. Results may differ for other models, precisions or serving stacks. The registered study named a different hosted model; this is a re-execution, not a replication of reported numbers.

2. **Protocol deviation.** The layout flag changes what the agent is told, identically for all arms. Absolute success rates depend on it.

3. **Underpowered primary endpoint.** Twelve seeds per arm cannot detect between-arm differences in M1@T24 of the size observed, given the between-seed variance.

4. **Exploratory statistics.** The registration placed inference out of scope; the tests reported here were chosen after the data were collected, although they are standard and applied uniformly to every endpoint.

5. **Monitoring during the campaign.** Pass counts were viewed while runs were in progress, contrary to the registered no-peeking rule. No rule or configuration was changed afterwards, which the repository history shows.

6. **Non-reproducible sampling.** Seeds are not passed to the model server, so reruns will differ; the logs are the evidence.

7. **One project, one sequence.** `taskcli` is small by design; generalization to other codebases and change patterns is untested.

8. **Shared base model.** The judge and generator are separate conversations with the same model.

8 Data and code availability

The repository is archived on Zenodo; release v1.0.0, which contains everything described here, is doi:10.5281/zenodo.23450109 (all versions: doi:10.5281/zenodo.23450108). The repository contains the frozen sequence and oracles, the harness, every main-campaign run (log, ledger, supersessions and the workspace's full git history) as per-arm archives, the excluded attempts as separate archives, the launcher logs, `results/metrics.csv` and `results/h1_events.csv`, and `results/isolation_checks.csv`. After unpacking the archives, `python -m harness.analyze` regenerates the metrics files byte-for-byte and `python -m harness.paper_numbers -isolation` regenerates every number in this paper, writing the macro file this paper includes and a claims table that maps each number to its source.

9 Conclusion

Run on a local open-weight model with every log public, executable requirement ledgers make an LLM coding agent produce the correct change more often and violate earlier requirements roughly half as often as a summary-only memory. The plain append-only ledger pays for this with 2.44× the tokens and with valid work rejected by stale tests, and its apparent perfection on the violation measures is produced by a scoring rule that excludes violations committed under stale-test pressure. Evidence-governed retirement keeps the protection at about 62% of the plain ledger's cost. Differences among the ledgered variants, and every comparison on the end-state primary endpoint, are within seed variance at this sample size.

AI assistance. The analysis scripts and a draft of this manuscript were prepared with the help of an AI assistant (Claude, Anthropic). The author reviewed and is responsible for all content.

References

[1] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *ICLR*, 2024.

[2] J. S. Park, J. C. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *UIST*, 2023.

[3] C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, and J. E. Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv:2310.08560*, 2023.

[4] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao. Reflexion: Language agents with verbal reinforcement learning. In *NeurIPS*, 2023.

[5] A. Maharana, D.-H. Lee, S. Tulyakov, M. Bansal, F. Barbieri, and Y. Fang. Evaluating very long-term conversational memory of LLM agents. In *ACL*, 2024.

[6] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang. Lost in the middle: How language models use long contexts. *TACL*, 12, 2024.

[7] D. Jiang, Y. Li, G. Li, Q. Li, and B. Li. HAGE: Harnessing agentic memory via RL-driven weighted graph evolution. *arXiv:2605.09942*, 2026.

[8] S. Yoo and M. Harman. Regression testing minimization, selection and prioritization: a survey. *Software Testing, Verification and Reliability*, 22(2), 2012.

[9] O. C. Z. Gotel and A. C. W. Finkelstein. An analysis of the requirements traceability problem. In *Proc. IEEE International Conference on Requirements Engineering*, 1994.

[10] B. Daniel, V. Jagannath, D. Dig, and D. Marinov. ReAssert: Suggesting repairs for broken unit tests. In *ASE*, 2009.

[11] K. Beck. *Test-Driven Development: By Example*. Addison-Wesley, 2002.

[12] D. Amodei, C. Olah, J. Steinhardt, P. Christiano, J. Schulman, and D. Mané. Concrete problems in AI safety. *arXiv:1606.06565*, 2016.

[13] B. A. Nosek, C. R. Ebersole, A. C. DeHaven, and D. T. Mellor. The preregistration revolution. *PNAS*, 115(11), 2018.

[14] Qwen Team. Qwen3 technical report. *arXiv:2505.09388*, 2025.